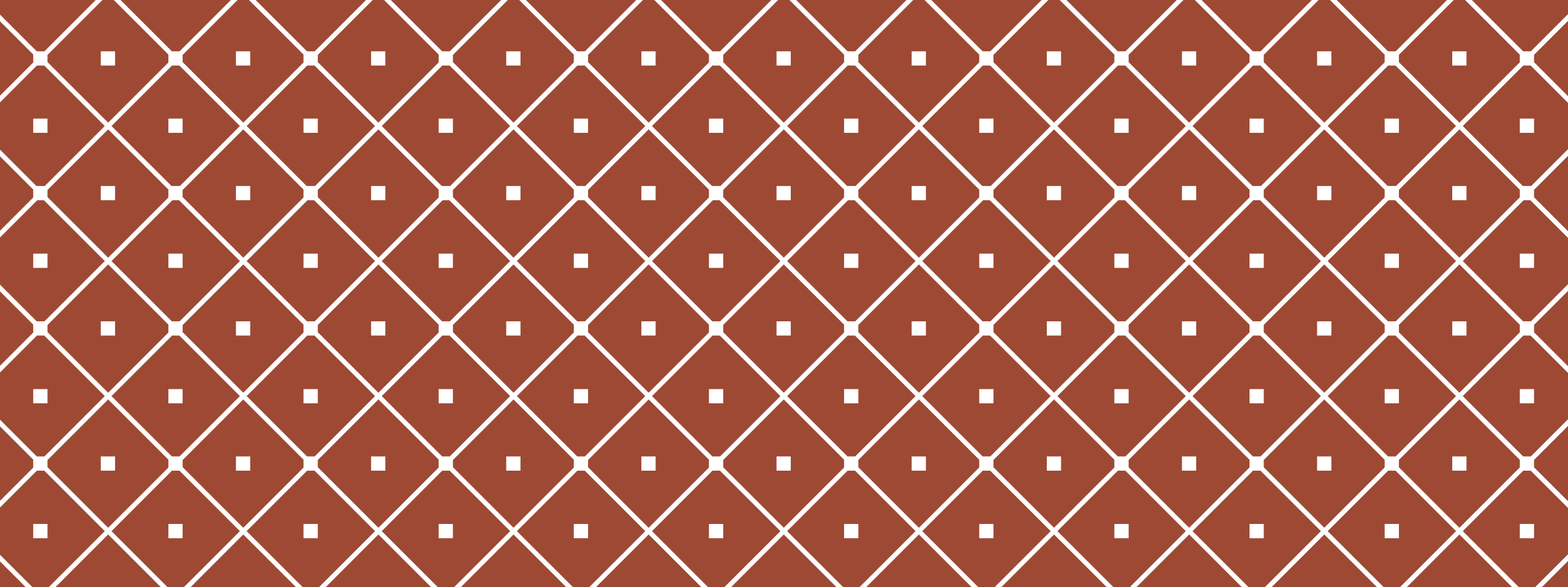




INTRODUCTION AU LANGAGE python™

PYTHON EST ...

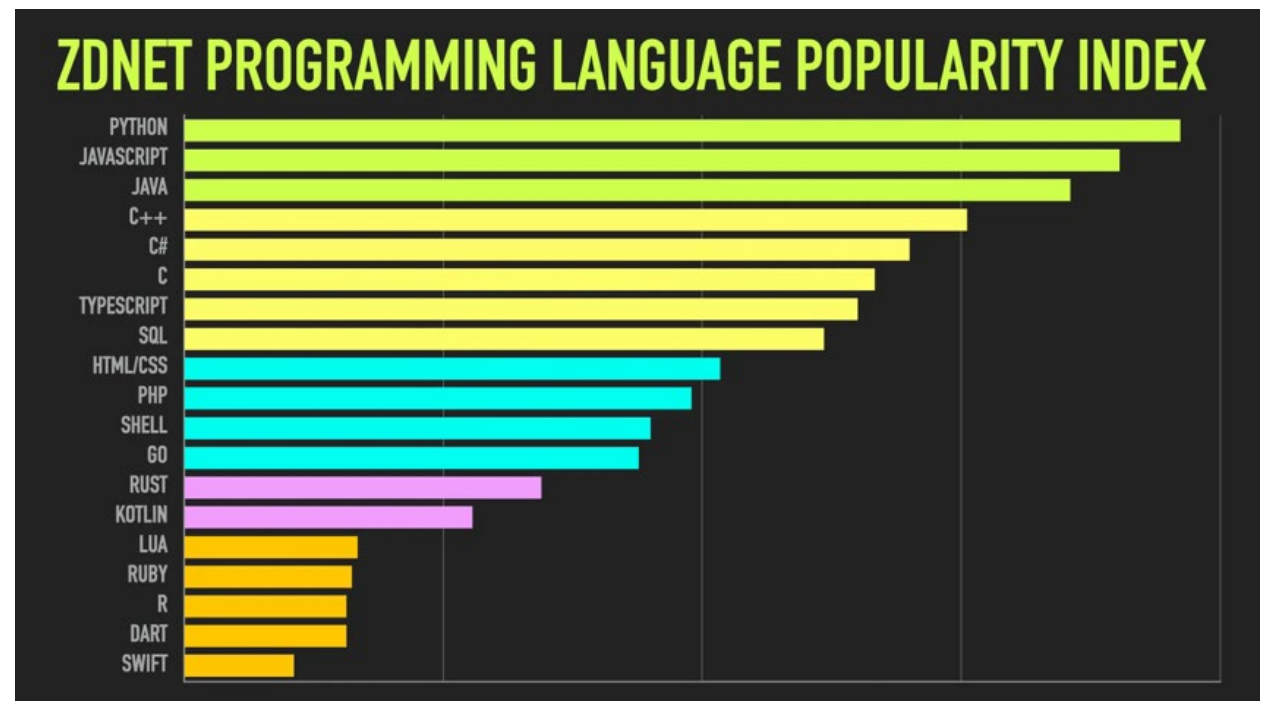
Simple

- Python est un langage simple, de haut-niveau et minimaliste par nature
- Sa nature proche du pseudo-code permet de mieux se concentrer sur le problème à résoudre plutôt que le langage

Facile à apprendre

Libre & Open source

Portable

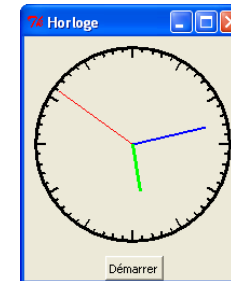
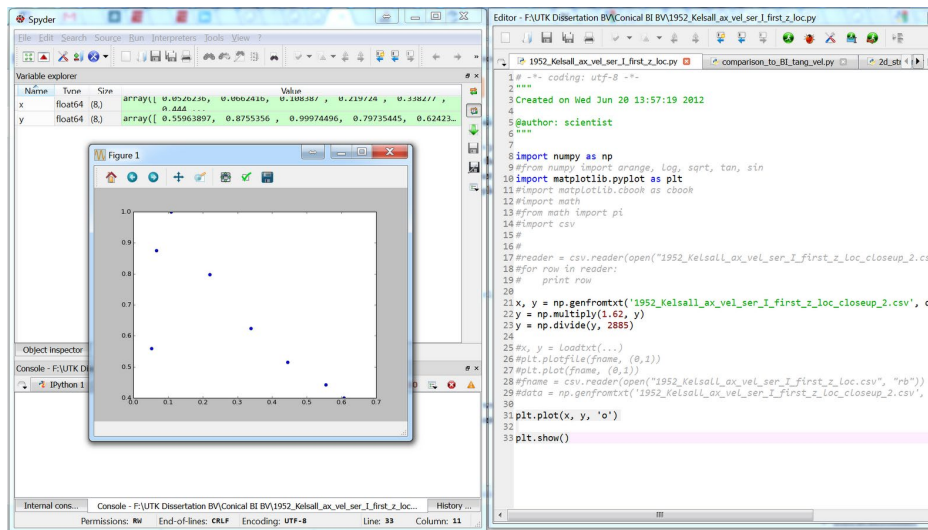


PYTHON EST ...

Un langage **interprété**

Extensible avec de nombreuses bibliothèques

- Réseau, image, son, cryptographie, calcul numérique – numpy, scipy, ... - etc. **Mais aussi** des bibliothèques graphiques - wxPython, tkinter, kivy, ...



HISTOIRE DE **Python**

Python a été conçu fin 1989 par Guido Van Rossum (code rendu public en 1991)

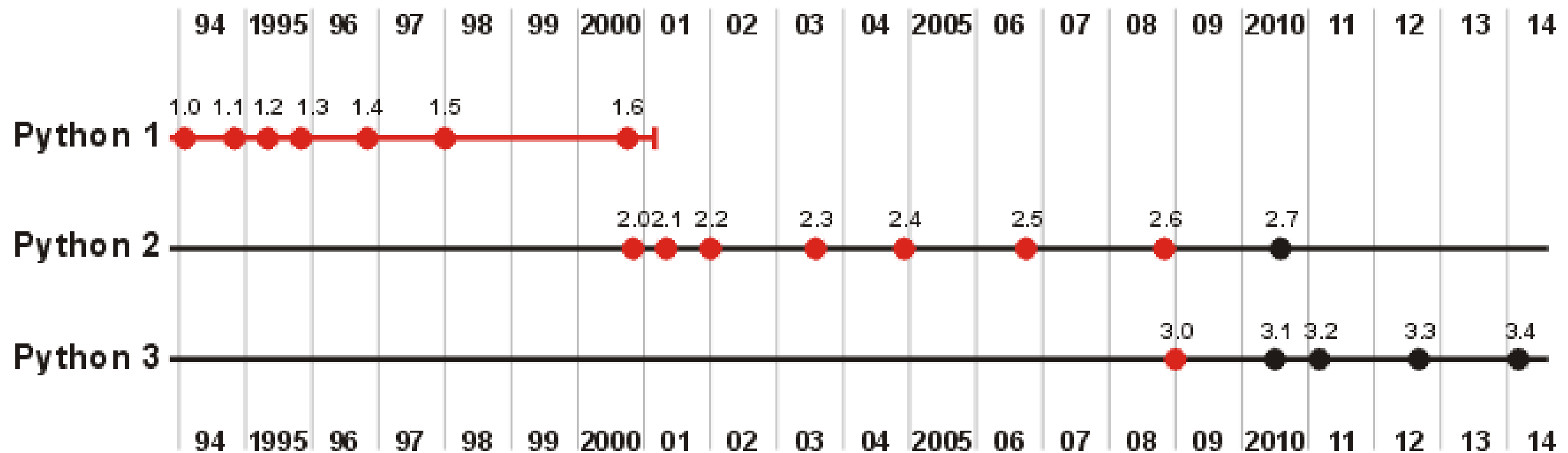
En Janvier 1994 sort Python 1.0. Avec la version 1.4, Python s'enrichit de nouvelles fonctionnalités dont notamment la gestion des nombres complexes.

Lance l'initiative "**Computer Programming for Everybody - CP4E** "
(financé par le DARPA)

- L'objectif est de rendre accessible la programmation au plus grand nombre notamment à ceux qui ne font pas de grandes études.



HISTOIRE DE PYTHON



HISTOIRE DE PYTHON

En 2008, apparition de Python 3.0 (a.k.a. “Py3k”)

- **Incompatible** avec les versions 2.x (**un premier problème**)

Aujourd’hui (**Avril 2026**) : 1 branche active incompatibles : Python **3.14.3**

Dernière version Python **2.7.18** (Avril 2020)

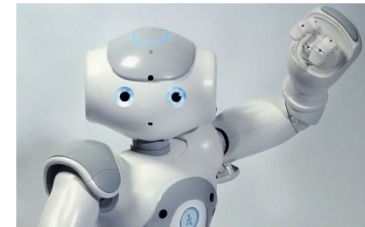
Les différentes améliorations et conventions du langage sont accessibles via les PEPs, **Python Enhancement Proposals** (<http://legacy.python.org/dev/peps>)

- PEP 8 : **Style Guide for Python Code** (<https://www.python.org/dev/peps/pep-0008>)
- PEP 20 : **The Zen of Python** (<https://www.python.org/dev/peps/pep-0020>)
- ...

PYTHON EST UTILISÉ ... (PARTOUT)

Dans des projets divers comme :

- blender
- SoftBank Robotics (Nao, Pepper, ...)
- micro Python pour micro-contrôleurs
- SciPy.org, (mathématiques et ingénierie)
- **Cybersécurité** et réseaux (<http://www.secdev.org/projects/scapy> ..., ...)



DES SHELLS INTERACTIFS EN LIGNE

<https://www.python.org>

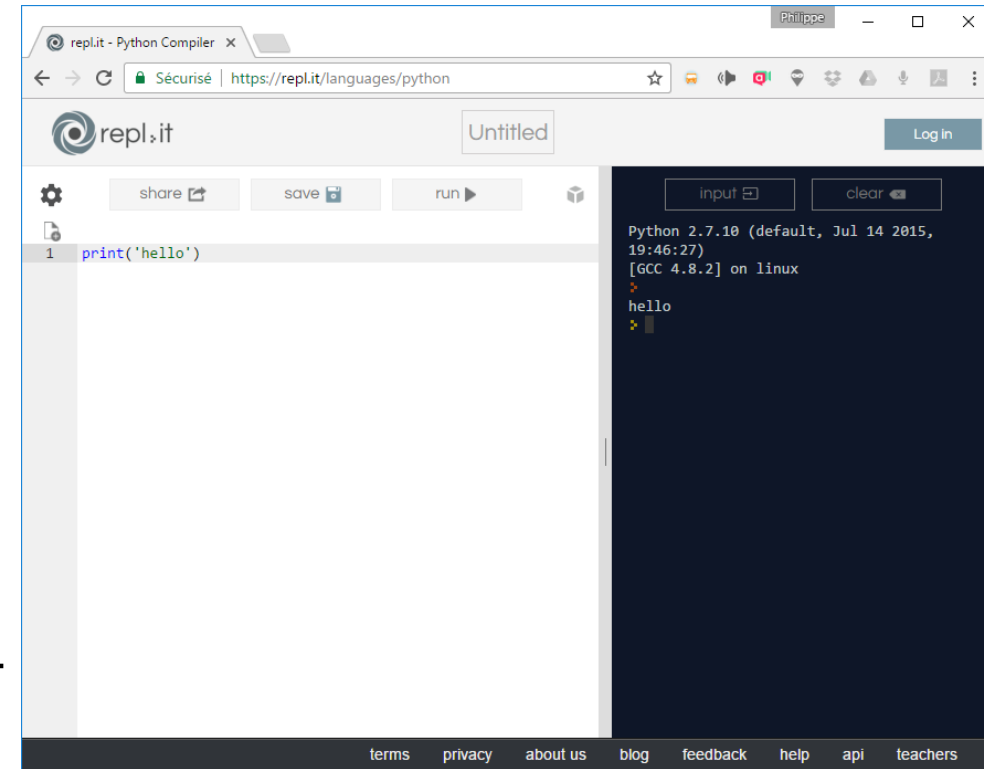
<https://repl.it/languages/python>

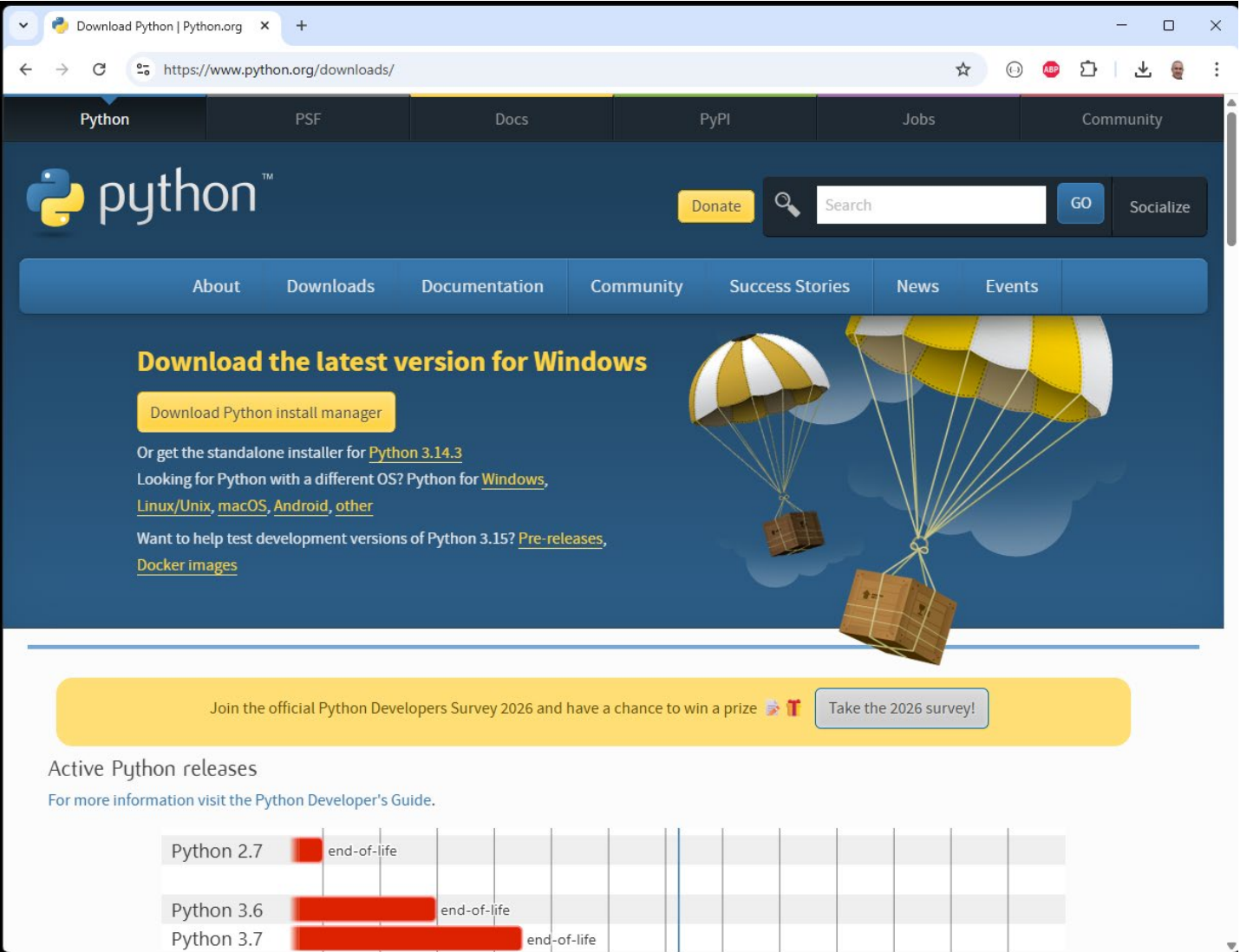
<https://repl.it/languages/python3>

<https://www.pythontutor.com/visualize.html#mode=edit>

<https://www.pythonanywhere.com>

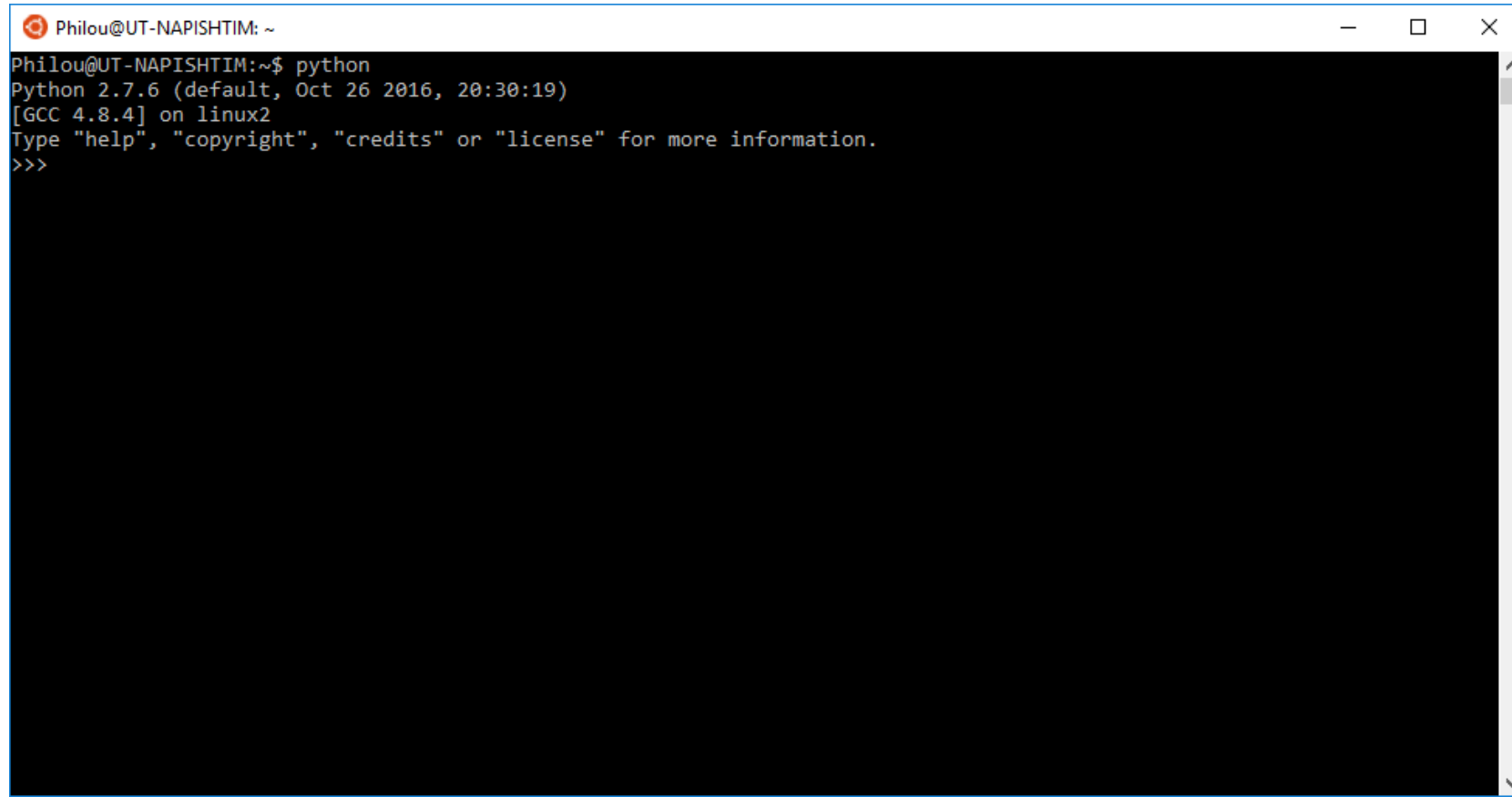
...





LANCER PYTHON

Il faut juste taper « **python** » ou « **python3** » (suivant les OS) dans un terminal

A terminal window with a black background and white text. The window title bar shows 'Philou@UT-NAPISHTIM: ~' and standard window controls. The terminal content shows the command 'python' being executed, followed by the Python 2.7.6 startup banner: 'Python 2.7.6 (default, Oct 26 2016, 20:30:19) [GCC 4.8.4] on linux2'. It then prompts the user to type 'help', 'copyright', 'credits', or 'license' for more information, and ends with the prompt '>>>'.

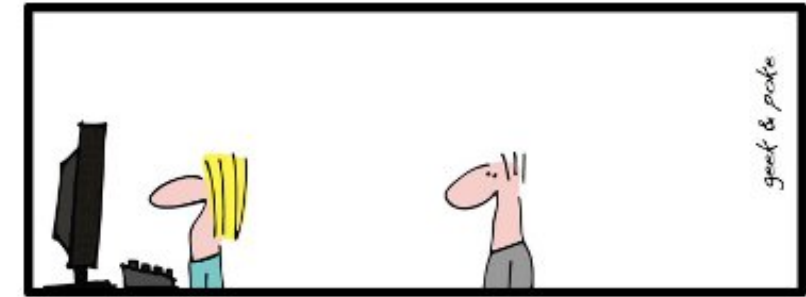
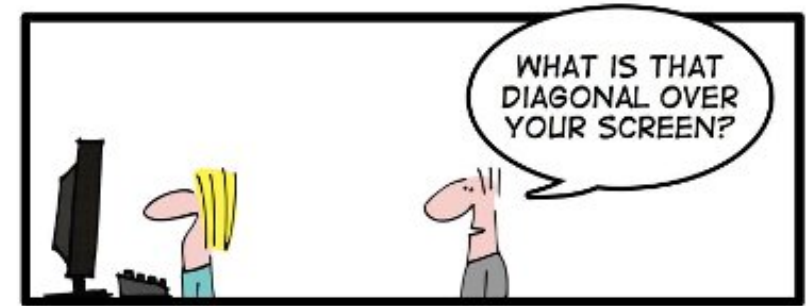
```
Philou@UT-NAPISHTIM: ~  
Philou@UT-NAPISHTIM:~$ python  
Python 2.7.6 (default, Oct 26 2016, 20:30:19)  
[GCC 4.8.4] on linux2  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

ALGORITHMIQUE & PYTHON

Python utilise les structures fondamentales de l'algorithmique

- La séquence
- La sélection (if, elif etc...)
- la répétition (while, for, ...)

<https://python.doctor/>



CHAPTER 2: PROGRAMMING

RULE 1: INDENT YOUR CODE
RULE 2: MAKE SURE IT HAS THE RIGHT COMPLEXITY

EXERCICE 0

Ouvrir un terminal, taper « **python** »

Taper « **import this** » puis « *touche entrée* »

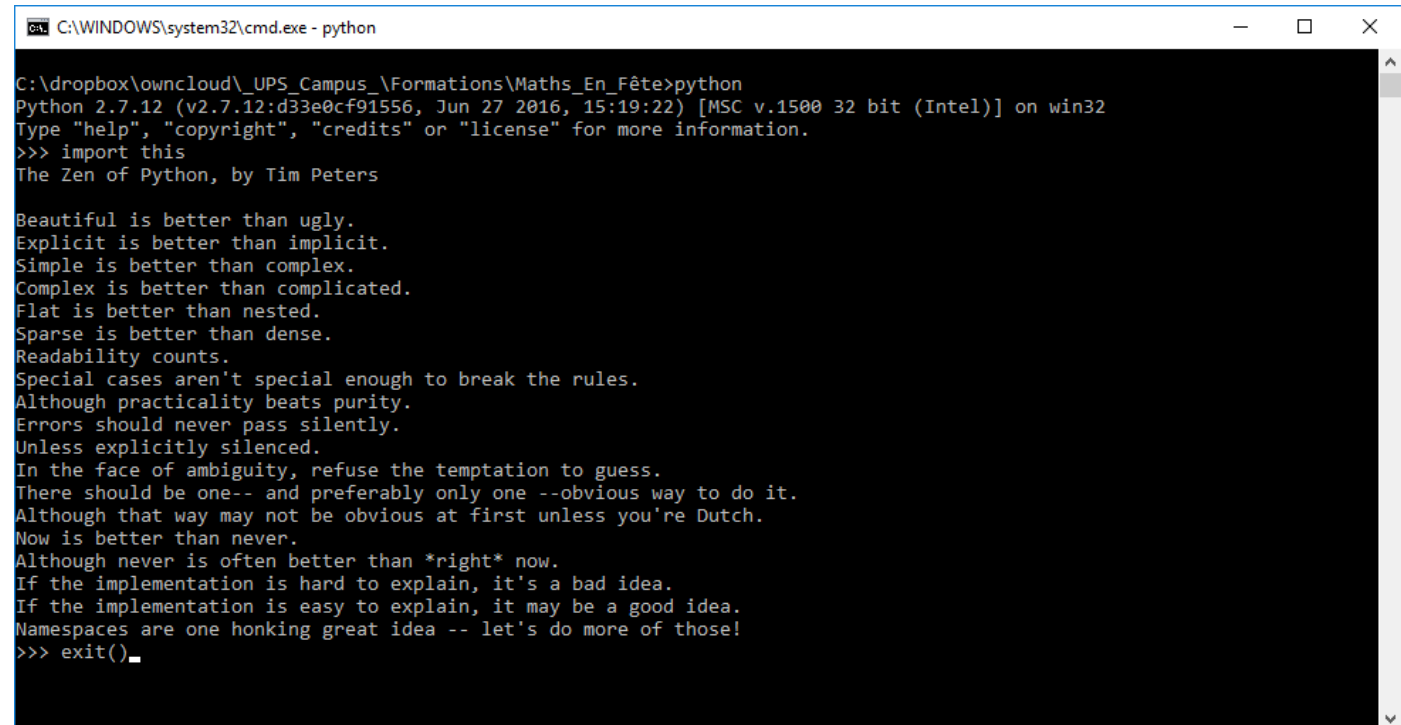
EXERCICE 0

Ouvrir un terminal, taper « **python** »

Taper « **import this** » puis « *touche entrée* »

Observer « l'easter egg PEP 20 »)

Taper « **exit()** » pour sortir



```
C:\WINDOWS\system32\cmd.exe - python

C:\dropbox\owncloud\UPS_Campus_Formations\Maths_En_Fête>python
Python 2.7.12 (v2.7.12:d33e0cf91556, Jun 27 2016, 15:19:22) [MSC v.1500 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>> exit()_
```

EN RÉALITÉ ...

On utilise un **éditeur de texte** (intégré comme thonny, spyder, IDLE, processing.py, ...) ou un éditeur « *basique* » (nano, notepad++, ...)

L'extension est **.py** (logique, non ?!)

Et on lance la commande : `python monfichier.py`

STRUCTURE D'UN PROGRAMME PYTHON

Des commentaires

Des structures ...

Des fonctions

Des répétitions

Des indentations

Structure de sélection

```
C:\dropbox\owncloud\_UPS_Campus\Formations\Maths_En_Fête\ex1.py - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramétrage Outils Macro Exécution Compléments Documents ?

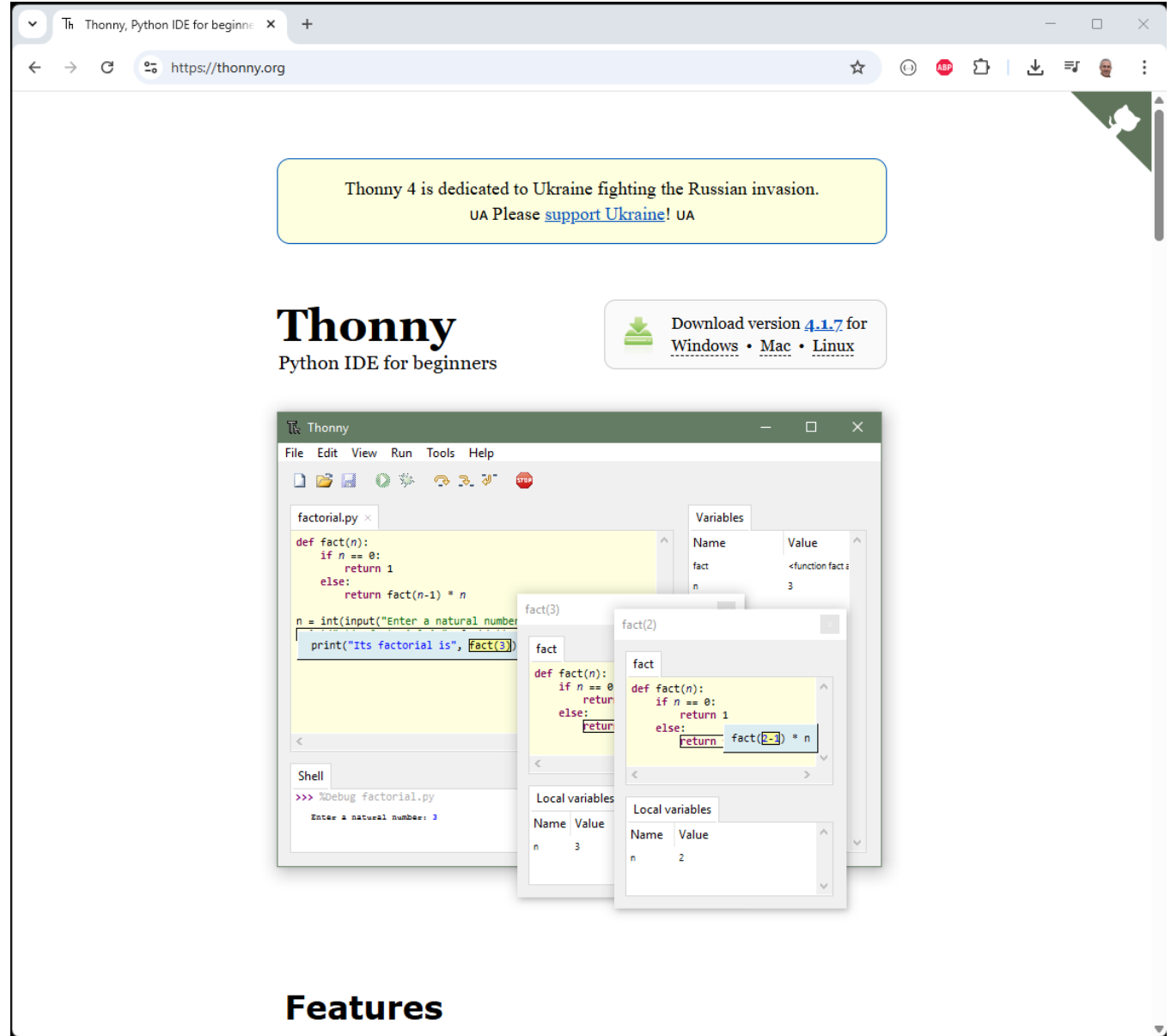
ex1.py
1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3  #
4  # Exercice 1 - encodage / décodage
5  #
6
7  # ddéfinir le dictinnaire
8  key = {'a':'n', 'b':'o', 'c':'p', 'd':'q', 'e':'r', 'f':'s', 'g':'t', 'h':'u', 'i':'v', 'j':'w', 'k':'x', 'l':'y', 'm':
9      'z', 'n':'a', 'o':'b', 'p':'c', 'q':'d', 'r':'e', 's':'f', 't':'g', 'u':'h', 'v':'i', 'w':'j', 'x':'k', 'y':'l', 'z':'m', ' ':' '}
10
11 def encode(txt):
12     txt_code = ''
13     # pour chaque lettre de l'alphabet, trouver son correspondant
14     for i in range(len(txt)):
15         txt_code = txt_code + key[txt[i]]
16     # renvoyer le résultat
17     return(txt_code)
18
19 # fonction principale
20 def main():
21     txt = raw_input('texte à encder : ')
22     txt_coded = encode(txt)
23     print "résultat: " + txt_coded
24
25 # utile pour le futur ;)
26 if __name__ == '__main__':
27     main()

Python file      length: 753  lines: 27      Ln: 20  Col: 41  Sel: 0 | 0      Unix (LF)      UTF-8      INS
```

Des particularités

UN ÉDITEUR : THONNY

<https://thonny.org>



The screenshot shows the Thonny website in a web browser. The browser's address bar displays <https://thonny.org>. The website features a yellow banner with the text: "Thonny 4 is dedicated to Ukraine fighting the Russian invasion. UA Please [support Ukraine!](#) UA". Below this, the word "Thonny" is prominently displayed in a large, bold, black serif font, with the tagline "Python IDE for beginners" underneath it. To the right, a green download icon is next to the text "Download version [4.1.7](#) for Windows • Mac • Linux".

Below the website content, a screenshot of the Thonny IDE interface is shown. The IDE window has a menu bar (File, Edit, View, Run, Tools, Help) and a toolbar. The main editor displays a file named "factorial.py" with the following Python code:

```
def fact(n):
    if n == 0:
        return 1
    else:
        return fact(n-1) * n

n = int(input("Enter a natural number: "))
print("Its factorial is", fact(n))
```

The IDE also includes a "Shell" window at the bottom with the command prompt showing the execution of the script: `>>> %Debug factorial.py` and the input `Enter a natural number: 3`. On the right side, there are two panels: "Variables" and "Local variables". The "Variables" panel shows a table with columns "Name" and "Value", containing entries for "fact" (a function object) and "n" (the value 3). The "Local variables" panel shows a table with columns "Name" and "Value", containing an entry for "n" (the value 2). Overlaid on the IDE are two smaller windows showing the function definition for "fact(3)" and "fact(2)", illustrating the recursive process.

Features

UN PREMIER EXERCICE

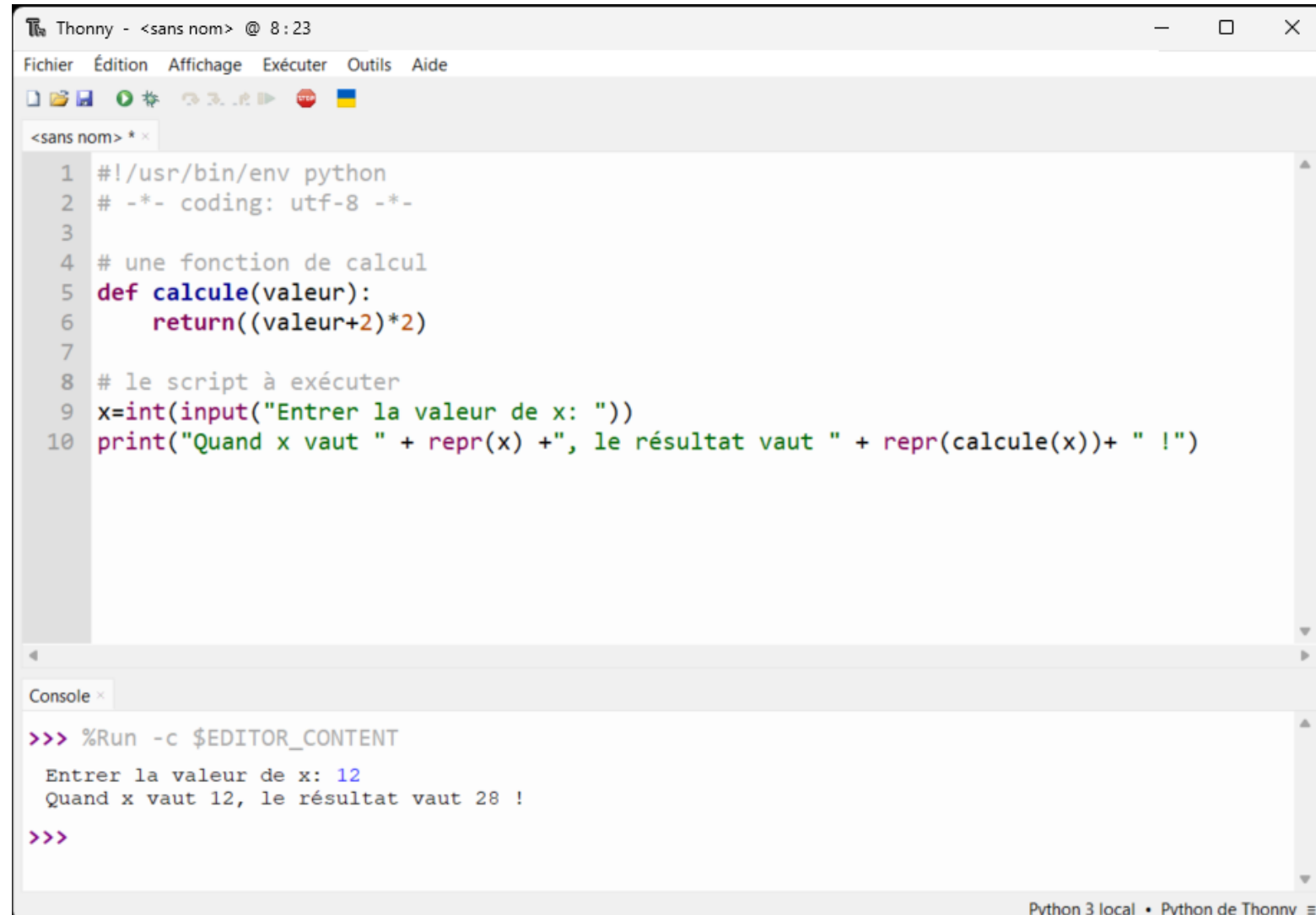
On veut simuler un programme comme ceux que l'on rencontre au collège :

- on choisit une valeur ;
- on ajoute 2 ;
- on multiplie par 2 ;
- on obtient le résultat final

EXERCICE DE CHAUFFE

On veut simuler un programme

- on choisit une valeur ;
- on ajoute 2 ;
- on multiplie par 2 ;
- on obtient le résultat final



The screenshot shows the Thonny Python IDE interface. The top menu bar includes 'Fichier', 'Édition', 'Affichage', 'Exécuter', 'Outils', and 'Aide'. Below the menu is a toolbar with icons for file operations and execution. The main editor window displays a Python script with the following code:

```
1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 # une fonction de calcul
5 def calcule(valeur):
6     return((valeur+2)*2)
7
8 # le script à exécuter
9 x=int(input("Entrer la valeur de x: "))
10 print("Quand x vaut " + repr(x) + ", le résultat vaut " + repr(calcule(x))+ " !")
```

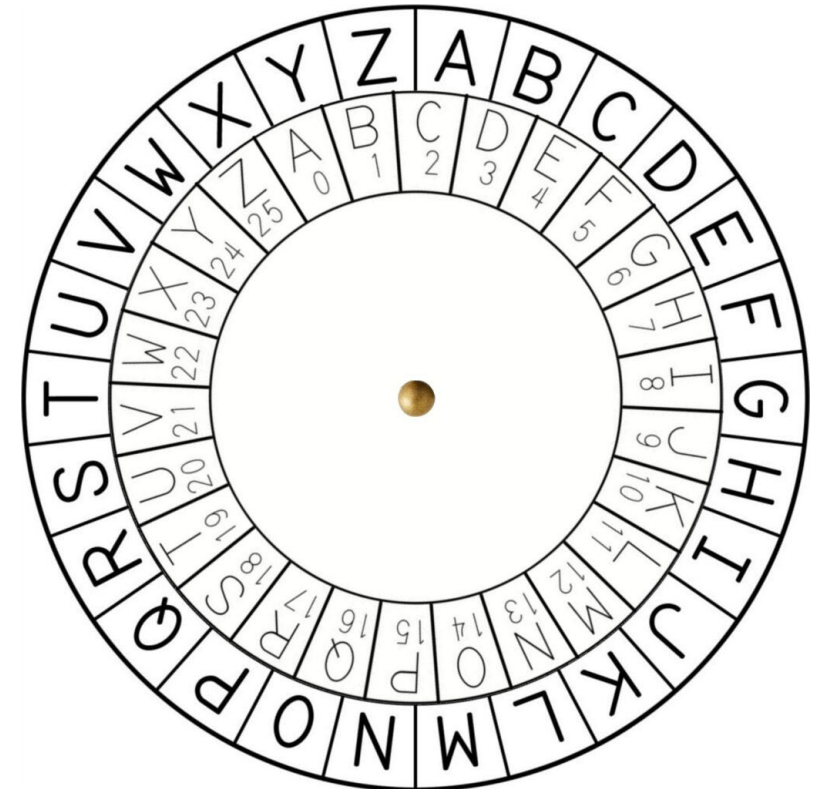
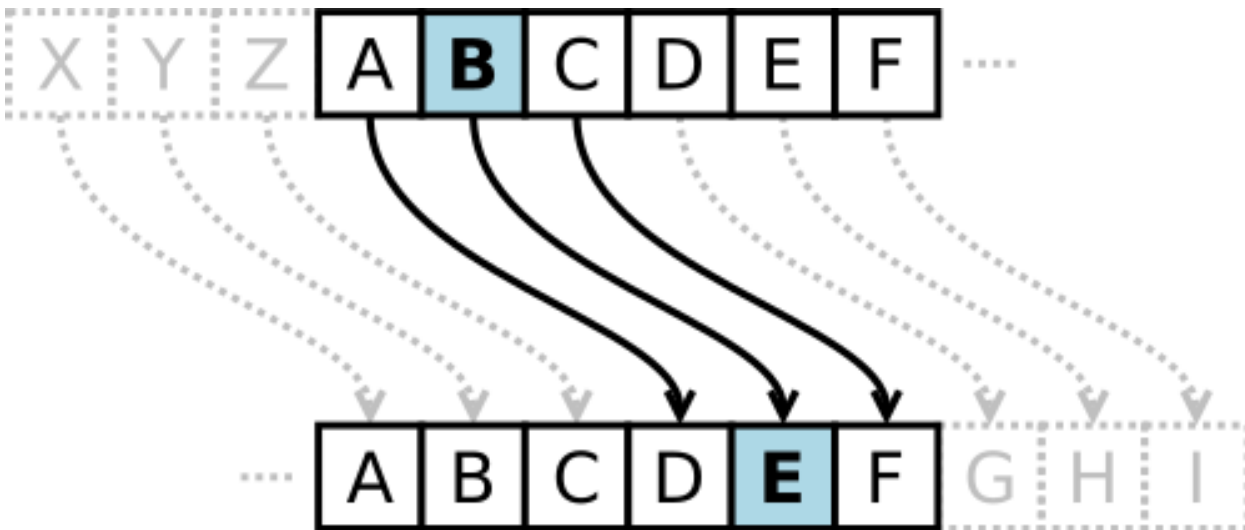
Below the editor is a console window titled 'Console'. It shows the execution of the script using the command `>>> %Run -c $EDITOR_CONTENT`. The output is:

```
Entrer la valeur de x: 12
Quand x vaut 12, le résultat vaut 28 !
>>>
```

The status bar at the bottom right indicates 'Python 3 local • Python de Thonny'.

UN PREMIER EXERCICE - CODE CESAR

Ecrire un programme en python qui encode/décode le code de CESAR (ROT13)



UN DEUXIEME EXERCICE - CODE OACI

Ecrire un programme python qui prend un mot ou texte en entrée et le transforme en « *alphabet OACI* » et permettre d'effectuer une sortie vocale du résultat

A – Alpha

B – Bravo

C – Charlie

D – Delta

E – Echo

F – Foxtrot

G – Golf

H – Hotel

I – India

J – Juliet

K – Kilo

L – Lima

M – Mike

N – November

O – Oscar

P – Papa

Q – Quebec

R – Romeo

S – Sierra

T – Tango

U – Uniform

V – Victor

W – Whiskey

X – X-Ray

Y – Yankee

Z – Zulu

LES LIBRAIRIES

De nombreuses librairies sont utilisées pour enrichir le langage : interfaces avec d'autres outils, spécificités par rapport à du matériel, tout pour permettre de manipuler, structurer, gérer vos logiciels et/ou matériel



LE GAME CHANGER : LE VIBE CODING

Vibe coding : pratique de développement assistée par l'IA

✶ Bonsoir, Phil.

je voudrais écrire un script python qui me permet de scanner les ports ouverts sur mon réseau local

+

Sonnet 4.6



je voudrais écrire un script python qui me permet de scanner les ports ouverts sur mon réseau local

Voici un script Python pour scanner les ports ouverts sur votre réseau local :

```
python

import socket
import ipaddress
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import datetime

# --- Configuration ---
NETWORK = "192.168.1.0/24" # Adaptez à votre réseau
PORTS = [21, 22, 23, 25, 53, 80, 110, 139, 143, 443, 445, 3306, 3389, 5900, 8080]
TIMEOUT = 0.5 # secondes
MAX_WORKERS = 100

# ---

PORT_NAMES = {
    21: "FTP", 22: "SSH", 23: "Telnet", 25: "SMTP", 53: "DNS",
    80: "HTTP", 110: "POP3", 139: "NetBIOS", 143: "IMAP", 443: "HTTPS",
    445: "SMB", 3306: "MySQL", 3389: "RDP", 5900: "VNC", 8080: "HTTP-alt"
}
```

Répondre...

+

Sonnet 4.6



ALLER PLUS LOIN ...

Beaucoup de ressources sur le web

De nombreux aspects du langage sont intéressants à étudier ... et les résultats sont spectaculaires 😊